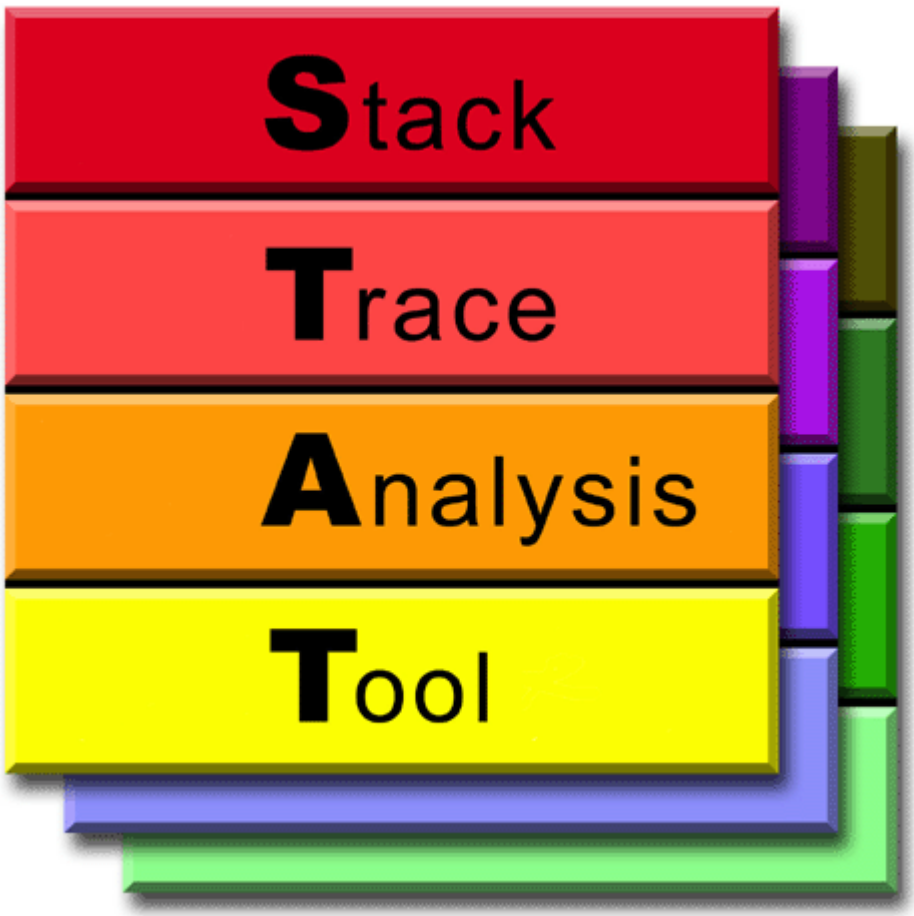
STAT: the Stack Trace Analysis Tool Quick Start Guide



Gregory L. Lee

Dorian C. Arnold

Dong H. Ahn

Bronis R. de Supinski

Matthew P. LeGendre

Barton P. Miller

Martin Schulz

STAT: the Stack Trace Analysis Tool Quick Start Guide

by Gregory L. Lee

by Dorian C. Arnold

by Dong H. Ahn

by Bronis R. de Supinski

by Matthew P. LeGendre

by Barton P. Miller

by Martin Schulz

Table of Contents

Disclaimer ..... **v**
Auspice v
License v
1. When To Use **1**
2. Using the STAT GUI **3**

Disclaimer

Auspice

This work performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under Contract DE-AC52-07NA27344.

License

Copyright (c) 2007-2020, Lawrence Livermore National Security, LLC.

Produced at the Lawrence Livermore National Laboratory

Written by Gregory Lee [lee218@llnl.gov], Dorian Arnold, Matthew LeGendre, Dong Ahn, Bronis de Supinski, Barton Miller, Martin Schulz, Niklas Nielson, Nicklas Bo Jensen, Jesper Nileson, and Sven Karlsson.

LLNL-CODE-750488.

All rights reserved.

This file is part of STAT. For details, see http://www.github.com/LLNL/STAT. Please also read STAT/LICENSE.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

Redistributions of source code must retain the above copyright notice, this list of conditions and the disclaimer below.

Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the disclaimer (as noted below) in the documentation and/or other materials provided with the distribution.

Neither the name of the LLNS/LLNL nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL LAWRENCE LIVERMORE NATIONAL SECURITY, LLC, THE U.S. DEPARTMENT OF ENERGY OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Additional BSD Notice

1. This notice is required to be provided under our contract with the U.S. Department of Energy (DOE). This work was produced at Lawrence Livermore National Laboratory under Contract No. DE-AC52-07NA27344 with the DOE.

2. Neither the United States Government nor Lawrence Livermore National Security, LLC nor any of their employees, makes any warranty, express or implied, or assumes any liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately-owned rights.

3. Also, reference herein to any specific commercial products, process, or services by trade name, trademark, manufacturer or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes.

Chapter 1. When To Use

STAT is most effective for diagnosing parallel applications that are hung (i.e., deadlock or livelock), however, its analysis can also be useful for debugging other errors. The outputted, merged stack traces indicate where in the code all of the processes are at a given point in time, giving the user insight into where the bug may be. The merged traces also form process equivalence classes, which can be used to select a subset of tasks to feed into a full-featured parallel debugger such as DDT or TotalView for root cause analysis.

Chapter 2. Using the STAT GUI

After launching the GUI via the **stat-gui** command you will first need to attach to the application via the **Attach** button. This will bring up the attach dialog (Figure 2-1). You will need to select the job launcher (i.e., **mpirun**, **srun**, or equivalent process). STAT will list processes owned by you on the localhost. If your job launcher process is on a remote host, you will need to enter that hostname in the remote host text entry box. Contact your local system administrator if you are not sure where to find your job launcher process. Once the appropriate process has been selected, click on the **Attach** in the lower right hand corner. STAT will then launch its daemons and gather an initial stack trace.

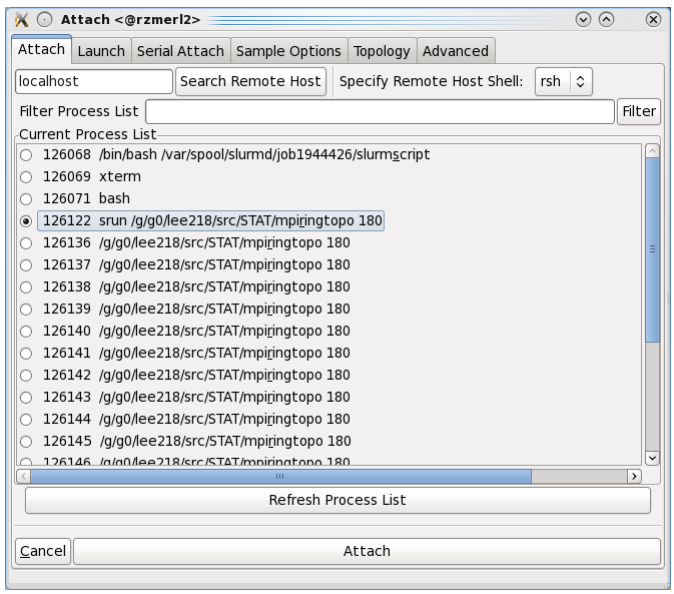


Figure 2-1. Use the attach dialog to select the job launcher process to attach to.

Once STAT has displayed the initial merged stack trace, in the form of a prefix tree, first you may want to look for common buggy patterns. This can be accomplished by using the analysis buttons in the toolbar across the top of the window. This includes operations to look for outliers such as processes with the shortest or longest stack trace or for the stack trace that was exhibited by the least or most processes. Note that these buttons are "traversal" buttons and they all initially operate on the full prefix tree. For example, the first click of the **[Shortest] Path** button will display the shortest path and subsequent clicks will display the next shortest path. Oftentimes bugs in parallel applications are triggered by a single or small subset of outliers in which case the **[Least] Tasks** button can quickly identify the outliers. Another common behavior is for a small subset of processes to be hung and the rest of the processes to be blocked in an MPI barrier or collective. In this case, the hung subset of tasks may have a shorter call path than the tasks in blocking in MPI, since the MPI implementation will usually be several frames deep. In this case, the **[Shortest] Path** can be useful.

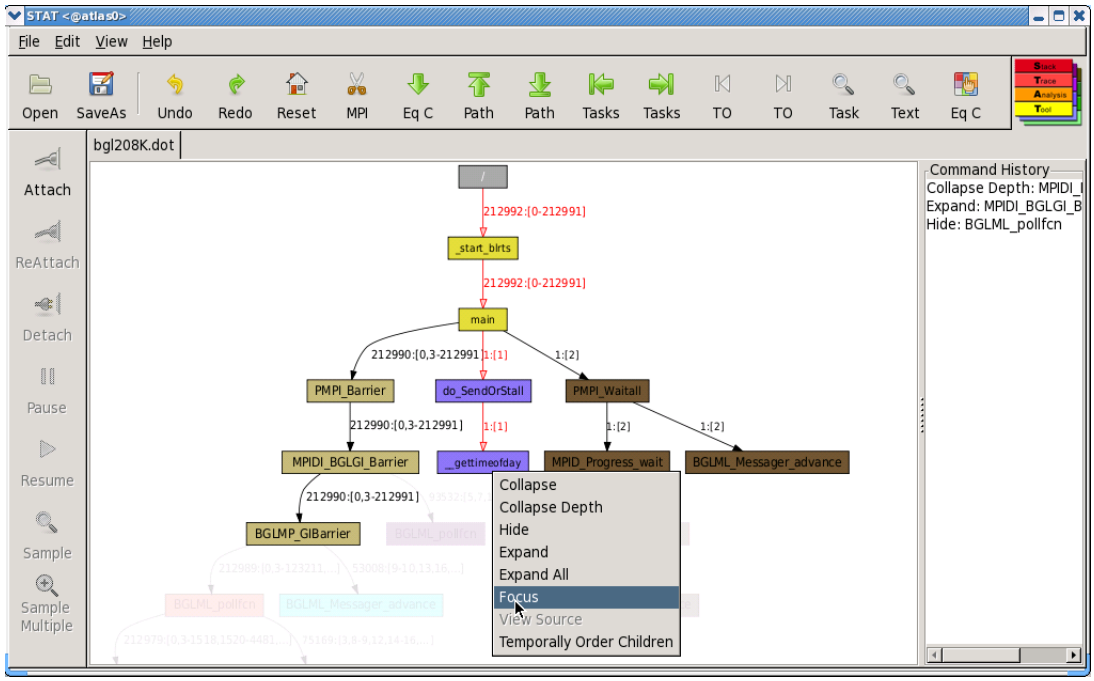


Figure 2-2. A screenshot of the STAT GUI.

Alternatively, you may wish to manually search through the stack traces. There are several buttons to aid in this process too. The **[Traverse] Eq C** will traverse the prefix tree, with each click traversing the down to the next point where there is a branch in equivalence classeses. There is also a **Search** button to search for specific MPI ranks, for stack frames with specified text, or for tasks running on specified nodes. Finally, left or right clicking on a node in the prefix tree gives you the option to expand or collapse the prefix tree. Note also that you can zoom in and out of the prefix tree using the options in the **View** menu or by using the scroll button on your mouse. You can also hold the left mouse button to "grab" the whitespace in the displayed prefix tree and move the focus around.

Another helpful button is the **Cut** button, which allows you to cut the tree at frames below a specified programming model's implementation. For example, **[Cut] MPI** will cut any frames below an MPI function call, thus allowing you to focus on application code as opposed to the MPI implementation stack frames. You can define your own programming model on the fly via the **Add Model** button. Default programming models are defined in the installation $prefix/etc/STAT/STATview_models.conf file or in the user $HOME/.STATview_models.conf file. Programming models are specified as regular expressions, using Python's **re** module syntax, and the **re.search** function is used in favor of **re.match**.

By default, the initial sample will gather stack traces at the granularity of function names. You can gather an additional sample with more detail, by clicking on the **Sample** button and selecting the **function and line** radio button. Note this typically requires that the code be compiled with the -g flag to get the appropriate debug information. After clicking **OK** a new prefix tree will be generated. By gathering stack traces with line number information, you may now associate stack traces back to the source code using the **View Source** button after clicking on a node in the prefix tree (Figure 2-3).

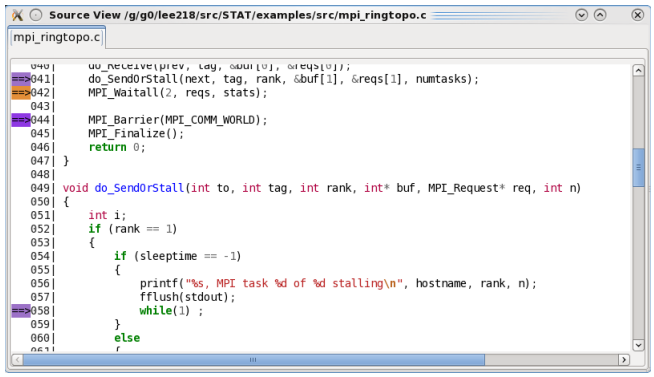


Figure 2-3. A screenshot of the source view window.

STAT was not intended to be a full-featured debugger, so you may ultimately need to employ another debugger such as DDT or TotalView for root cause analysis. STAT includes an interface to launch either of these debuggers (where available) on a subset of the MPI tasks based on the equivalence classes that STAT identifies. This interface can be accessed through the **[Identify] Eq C** button in the upper right hand corner of the window. In order to allow the other debuggers to attach, STAT will first detach itself from the application. Pinpointing a bug may require several iterations of running STAT on the entire application and running a full-featured debugger on a subset. After detaching a full-featured debugger, you can quickly attach to your application again with the **ReAttach** button.